

AuditPlatform

Rapport d'analyse — plateforme-audit-gitlab

Généré le 16/04/2026 à 15:45

■ Informations du projet

Nom du projet	plateforme-audit-gitlab
URL du projet	yosrchiha01/plateforme-audit-gitlab
Branche analysée	main
Date de l'analyse	16/04/2026 15:23
Modèle LLM	llama-3.3-70b-versatile
Analyse OWASP	Activée

■ Scores

Qualité	74	74/100
Sécurité	40	40/100
Performance	63	63/100

■ ■ Vulnérabilités (84)

Complexité
Sévérité : HAUTE
Fichier : backend/app/routes/auth.py — ligne 0
Correction : Réduire la complexité de la fonction get_current_user en la décomposant en plusieurs fonctions plus petites et en utilisant des variables de classe pour stocker les valeurs de configuration
Complexité
Sévérité : HAUTE

Fichier : backend/app/routes/depots.py — ligne 0

Correction : Réduire la complexité de la fonction compare_depot en la décomposant en plusieurs fonctions plus petites et en utilisant des variables de classe pour stocker les valeurs de configuration

A02 — CRYPTOGRAPHIC FAILURES

Sévérité : HAUTE

Fichier : backend/app/config/database.py — ligne 1

Correction : Utiliser une méthode de hachage sécurisée comme bcrypt ou Argon2 pour stocker les mots de passe.

A02 — CRYPTOGRAPHIC FAILURES

Sévérité : HAUTE

Fichier : backend/app/config/mail.py — ligne 1

Correction : Utiliser une méthode de chiffrement sécurisée comme TLS pour protéger les données sensibles transmises.

A02 — CRYPTOGRAPHIC FAILURES

Sévérité : HAUTE

Fichier : backend/app/config/settings.py — ligne 1

Correction : Utiliser une méthode de chiffrement sécurisée comme TLS pour protéger les données sensibles transmises.

A02 — CRYPTOGRAPHIC FAILURES

Sévérité : HAUTE

Fichier : backend/app/core/security.py — ligne 1

Correction : Utiliser une méthode de hachage sécurisée comme bcrypt ou Argon2 pour stocker les mots de passe.

A07 — IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : backend/app/routes/auth.py — ligne 1

Correction : Utiliser une méthode de hachage sécurisée comme bcrypt ou Argon2 pour stocker les mots de passe.

A10 — SSRF (SERVER-SIDE REQUEST FORGERY)

Sévérité : HAUTE

Fichier : backend/app/routes/depots.py — ligne 1

Correction : Utiliser une liste blanche des domaines autorisés pour éviter les appels HTTP vers des URLs contrôlées par l'utilisateur.

A10 — SSRF (SERVER-SIDE REQUEST FORGERY)

Sévérité : HAUTE

Fichier : backend/app/services/gitlab_service.py — ligne 1

Correction : Utiliser une liste blanche des domaines autorisés pour éviter les appels HTTP vers des URLs contrôlées par l'utilisateur.

A02 — CRYPTOGRAPHIC FAILURES

Sévérité : HAUTE

Fichier : backend/send_test_mail.py — ligne 1

Correction : Utiliser une méthode de chiffrement sécurisée comme TLS pour protéger les données sensibles transmises.

A10 — SSRF (SERVER-SIDE REQUEST FORGERY)

Sévérité : HAUTE

Fichier : frontend/app/add-repository/page.tsx — ligne 1

Correction : Utiliser une liste blanche des domaines autorisés pour éviter les appels HTTP vers des URLs contrôlées par l'utilisateur.

Boucle $O(n^2)$

Sévérité : HAUTE

Fichier : backend/app/routes/depots.py — ligne 43

Correction : Utiliser une boucle avec un index ou une structure de données pour améliorer la complexité

Boucle $O(n^2)$

Sévérité : HAUTE

Fichier : backend/app/routes/depots.py — ligne 63

Correction : Utiliser une boucle avec un index ou une structure de données pour améliorer la complexité

SOLID-O

Sévérité : HAUTE

Fichier : backend/app/routes/auth.py — ligne 24

Correction : La classe User est trop grande et contient trop de méthodes. Il est préférable de la décomposer en plusieurs classes plus petites et plus spécialisées.

Requêtes N+1

Sévérité : HAUTE

Fichier : frontend/app/depots/page.tsx — ligne 24

Correction : Utiliser des jointures pour réduire le nombre de requêtes

Appels API externes à l'intérieur de boucles

Sévérité : HAUTE

Fichier : frontend/app/difference/page.tsx — ligne 14

Correction : Utiliser un mécanisme de cache pour les données rarement modifiées

Boucles imbriquées avec complexité $O(n^2)$

Sévérité : HAUTE

Fichier : frontend/app/profile/page.tsx — ligne 24

Correction : Utiliser des boucles avec complexité $O(n)$ ou moins

A07 - IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : frontend/app/reset-password/page.tsx — ligne 1

Correction : Utiliser une bibliothèque de gestion d'authentification sécurisée, comme Next-Auth, et configurer les jetons JWT avec une expiration et un algorithme sécurisé.

SOLID-O

Sévérité : HAUTE

Fichier : frontend/next.config.ts — ligne 0

Correction : Le fichier next.config.ts n'est pas utilisé dans le code. Il est recommandé de le supprimer ou de le remplacer par un fichier de configuration plus pertinent.

Complexité

Sévérité : MOYENNE

Fichier : backend/app/config/database.py — ligne 0

Correction : Réduire la complexité de la fonction get_db en la décomposant en plusieurs fonctions plus petites

Complexité

Sévérité : MOYENNE

Fichier : backend/app/services/gitlab_service.py — ligne 0

Correction : Réduire la complexité de la fonction get_projects en la décomposant en plusieurs fonctions plus petites et en utilisant des variables de classe pour stocker les valeurs de configuration

SELECT*

Sévérité : MOYENNE

Fichier : backend/app/config/database.py — ligne 14

Correction : Utiliser les colonnes nécessaires au lieu de SELECT *

Boucle O(n)

Sévérité : MOYENNE

Fichier : backend/app/routes/gitlab.py — ligne 6

Correction : Utiliser un index ou une structure de données pour améliorer la complexité

Boucle O(n)

Sévérité : MOYENNE

Fichier : backend/app/services/gitlab_service.py — ligne 6

Correction : Utiliser un index ou une structure de données pour améliorer la complexité

Boucle O(n)

Sévérité : MOYENNE

Fichier : backend/send_test_mail.py — ligne 14

Correction : Utiliser un index ou une structure de données pour améliorer la complexité

Boucle $O(n)$

Sévérité : MOYENNE

Fichier : frontend/app/add-repository/page.tsx — ligne 24

Correction : Utiliser un index ou une structure de données pour améliorer la complexité

Boucle $O(n)$

Sévérité : MOYENNE

Fichier : frontend/app/dashboard/page.tsx — ligne 64

Correction : Utiliser un index ou une structure de données pour améliorer la complexité

Docstring manquant

Sévérité : MOYENNE

Fichier : backend/app/config/database.py — ligne 14

Correction : Ajouter une docstring pour expliquer le rôle de la fonction `get_db()`

Commentaire absent

Sévérité : MOYENNE

Fichier : backend/app/config/mail.py — ligne 1

Correction : Ajouter un commentaire pour expliquer la configuration mail

Commentaire absent

Sévérité : MOYENNE

Fichier : backend/app/routes/depos.py — ligne 1

Correction : Ajouter un commentaire pour expliquer la fonction `get_db()`

Commentaire absent

Sévérité : MOYENNE

Fichier : backend/app/services/gitlab_service.py — ligne 1

Correction : Ajouter un commentaire pour expliquer la fonction `get_projects()`

SOLID-S

Sévérité : MOYENNE

Fichier : backend/app/config/database.py — ligne 14

Correction : La méthode `get_db()` est appelée à plusieurs reprises dans l'application. Il est préférable de la factoriser en une seule instance pour éviter les problèmes de concurrence.

Gestion erreurs

Sévérité : MOYENNE

Fichier : backend/app/routes/depots.py — ligne 34

Correction : La méthode `compare_depot()` ne gère pas les exceptions correctement. Il est préférable d'utiliser un bloc `try-except` pour gérer les erreurs et fournir des informations utiles à l'utilisateur.

Dependency Inversion

Sévérité : MOYENNE

Fichier : backend/app/services/gitlab_service.py — ligne 10

Correction : La classe `get_projects()` dépend directement de l'implémentation concrète de la classe `requests`. Il est préférable de l'inverser pour utiliser une interface plus abstraite.

Complexité

Sévérité : MOYENNE

Fichier : frontend/app/depots/page.tsx — ligne 0

Correction : Réduire la complexité de la fonction `fetchDepots` en utilisant des fonctions plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : frontend/app/depots/page.tsx — ligne 0

Correction : Réduire la complexité de la fonction `handleDelete` en utilisant des fonctions plus petites et plus faciles à comprendre.

Complexité

Sévérité : MOYENNE

Fichier : frontend/app/difference/page.tsx — ligne 0

Correction : Réduire la complexité de la fonction useSearchParams en utilisant des fonctions plus petites et plus faciles à comprendre.

Complexité

Sévérité : MOYENNE

Fichier : frontend/app/forgot-password/page.tsx — ligne 0

Correction : Réduire la complexité de la fonction handleSubmit en utilisant des fonctions plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : frontend/app/lib/api.ts — ligne 0

Correction : Réduire la complexité de la fonction loginUser en utilisant des fonctions plus petites et plus faciles à comprendre.

Complexité

Sévérité : MOYENNE

Fichier : frontend/app/login/page.tsx — ligne 0

Correction : Réduire la complexité de la fonction handleLogin en utilisant des fonctions plus petites et plus faciles à comprendre.

Complexité

Sévérité : MOYENNE

Fichier : frontend/app/profile/page.tsx — ligne 0

Correction : Réduire la complexité de la fonction handleLogout en utilisant des fonctions plus petites et plus faciles à comprendre.

Complexité

Sévérité : MOYENNE

Fichier : frontend/app/register/page.tsx — ligne 0

Correction : Réduire la complexité de la fonction handleRegister en utilisant des fonctions plus petites et plus faciles à comprendre.

SELECT*

Sévérité : MOYENNE

Fichier : frontend/app/depots/page.tsx — ligne 12

Correction : Utiliser les colonnes nécessaires au lieu de SELECT *

Fichiers ouverts sans bloc try/finally

Sévérité : MOYENNE

Fichier : frontend/app/forgot-password/page.tsx — ligne 14

Correction : Utiliser un bloc try/finally pour fermer le fichier

Recherche linéaire O(n)

Sévérité : MOYENNE

Fichier : frontend/app/login/page.tsx — ligne 24

Correction : Utiliser un index/dictionnaire pour améliorer les performances

Fichiers ouverts sans bloc try/finally

Sévérité : MOYENNE

Fichier : frontend/app/register/page.tsx — ligne 14

Correction : Utiliser un bloc try/finally pour fermer le fichier

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/app/depots/page.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le rôle de la fonction DepotsPage

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/app/depots/page.tsx — ligne 15

Correction : Ajouter un commentaire pour expliquer la logique de la fonction filtered

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/app/difference/page.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le rôle de la fonction DifferencePage

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/app/forgot-password/page.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le rôle de la fonction ForgotPassword

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/app/layout.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le rôle de la fonction RootLayout

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/app/lib/api.ts — ligne 1

Correction : Ajouter une docstring pour expliquer le rôle de la fonction loginUser

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/app/login/page.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le rôle de la fonction LoginPage

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/app/profile/page.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le rôle de la fonction ProfilePage

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/app/register/page.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le rôle de la fonction RegisterPage

Complexité

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 0

Correction : Réduire la complexité de la fonction handleSubmit en utilisant des fonctions plus petites et plus faciles à comprendre.

A08 - DATA INTEGRITY FAILURES

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 34

Correction : Valider les données entrantes avant de les utiliser dans la requête de réinitialisation du mot de passe.

SELECT*

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 24

Correction : Utiliser les colonnes nécessaires au lieu de SELECT *

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 15

Correction : Ajouter une docstring pour expliquer le rôle de la fonction handleSubmit

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 34

Correction : Ajouter un commentaire pour expliquer l'algorithme de vérification des mots de passe

SOLID-S

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 24

Correction : La méthode handleSubmit contient plusieurs responsabilités. Il est recommandé de la décomposer en méthodes plus petites et plus spécifiques.

Lisibilité

Sévérité : FAIBLE

Fichier : backend/app/schemas/depot.py — ligne 0

Correction : Utiliser des noms de variables plus explicites et des commentaires pour expliquer le code

Lisibilité

Sévérité : FAIBLE

Fichier : backend/app/schemas/user.py — ligne 0

Correction : Utiliser des noms de variables plus explicites et des commentaires pour expliquer le code

Lisibilité

Sévérité : FAIBLE

Fichier : backend/send_test_mail.py — ligne 0

Correction : Utiliser des noms de variables plus explicites et des commentaires pour expliquer le code

Lisibilité

Sévérité : FAIBLE

Fichier : frontend/app/add-repository/page.tsx — ligne 0

Correction : Utiliser des noms de variables plus explicites et des commentaires pour expliquer le code

Lisibilité

Sévérité : FAIBLE

Fichier : frontend/app/components/Button.tsx — ligne 0

Correction : Utiliser des noms de variables plus explicites et des commentaires pour expliquer le code

Lisibilité

Sévérité : FAIBLE

Fichier : frontend/app/components/Input.tsx — ligne 0

Correction : Utiliser des noms de variables plus explicites et des commentaires pour expliquer le code

Lisibilité

Sévérité : FAIBLE

Fichier : frontend/app/dashboard/page.tsx — ligne 0

Correction : Utiliser des noms de variables plus explicites et des commentaires pour expliquer le code

Nommage ambigu

Sévérité : FAIBLE

Fichier : backend/app/routes/auth.py — ligne 1

Correction : Renommer la fonction `get_db()` en `get_database_session()` pour refléter son rôle

Duplication

Sévérité : FAIBLE

Fichier : frontend/app/depots/page.tsx — ligne 0

Correction : Supprimer la duplication de code dans la fonction `handleDelete` en utilisant des fonctions plus petites et plus faciles à comprendre.

Lisibilité

Sévérité : FAIBLE

Fichier : frontend/app/depots/page.tsx — ligne 0

Correction : Utiliser des noms de variables plus significatifs et plus faciles à comprendre, par exemple, `'depots'` au lieu de `'d'`.

Lisibilité

Sévérité : FAIBLE

Fichier : frontend/app/layout.tsx — ligne 0

Correction : Utiliser des noms de variables plus significatifs et plus faciles à comprendre, par exemple, `'metadata'` au lieu de `'metadata'`.

Lisibilité

Sévérité : FAIBLE

Fichier : frontend/app/page.tsx — ligne 0

Correction : Utiliser des noms de variables plus significatifs et plus faciles à comprendre, par exemple, 'children' au lieu de 'children'.

Connexions non fermées

Sévérité : FAIBLE

Fichier : frontend/app/depots/page.tsx — ligne 34

Correction : Fermer la connexion après utilisation

Gestion de la mémoire et des ressources

Sévérité : FAIBLE

Fichier : frontend/app/layout.tsx — ligne 10

Correction : Utiliser un gestionnaire de contexte pour les ressources

Nommage ambigu

Sévérité : FAIBLE

Fichier : frontend/app/depots/page.tsx — ligne 25

Correction : Renommer la variable filtered en quelque chose de plus explicite

Lisibilité

Sévérité : FAIBLE

Fichier : frontend/app/reset-password/page.tsx — ligne 0

Correction : Utiliser des noms de variables plus significatifs, par exemple, au lieu de 'e' pour l'événement, utiliser 'event' ou 'handleSubmitEvent'.

Maintenabilité

Sévérité : FAIBLE

Fichier : frontend/next.config.ts — ligne 0

Correction : Considérer l'utilisation d'un fichier de configuration séparé pour les options de Next.js, cela améliorera la lisibilité et la maintenabilité du code.

A05 - SECURITY MISCONFIGURATION

Sévérité : FAIBLE

Fichier : frontend/next.config.ts — ligne 1

Correction : Activer la configuration de sécurité pour Next.js, notamment la protection contre les attaques de brute force et la validation des en-têtes HTTP de sécurité.

Boucle $O(n^2)$

Sévérité : FAIBLE

Fichier : frontend/app/reset-password/page.tsx — ligne 34

Correction : Utiliser un algorithme de recherche plus efficace ou un index/dictionnaire

Mémoire

Sévérité : FAIBLE

Fichier : frontend/app/reset-password/page.tsx — ligne 44

Correction : Utiliser un gestionnaire de contexte pour les fichiers ouverts

Ressource

Sévérité : FAIBLE

Fichier : frontend/next.config.ts — ligne 0

Correction : Utiliser un gestionnaire de contexte pour les connexions réseau

Nommage ambigu

Sévérité : FAIBLE

Fichier : frontend/next.config.ts — ligne 4

Correction : Considérer un nom plus explicite pour la variable nextConfig

Gestion erreurs

Sévérité : FAIBLE

Fichier : frontend/app/reset-password/page.tsx — ligne 43

Correction : La gestion des erreurs dans la méthode handleSubmit est trop générique. Il est recommandé d'utiliser des exceptions spécifiques pour les différents types d'erreurs.

■ Recommandations

1. Réduire la complexité du code

Le code est trop complexe et difficile à comprendre. Il est recommandé de le décomposer en plusieurs fonctions plus petites et de l'expliquer à l'aide de commentaires.

2. Utiliser des variables de classe

Les variables de classe sont utilisées pour stocker les valeurs de configuration. Il est recommandé de les utiliser pour stocker les valeurs de configuration au lieu de les passer en paramètre des fonctions.

3. Utiliser des noms de variables plus explicites

Les noms de variables sont trop courts et ne sont pas explicites. Il est recommandé de les utiliser pour expliquer le code et de les rendre plus lisibles.

4. Utiliser des commentaires

Les commentaires sont utilisés pour expliquer le code. Il est recommandé de les utiliser pour expliquer le code et de les rendre plus lisibles.

5. Utiliser des méthodes de hachage sécurisées

Utiliser des méthodes de hachage sécurisées comme bcrypt ou Argon2 pour stocker les mots de passe et éviter les attaques par force brute.

6. Utiliser des méthodes de chiffrement sécurisées

Utiliser des méthodes de chiffrement sécurisées comme TLS pour protéger les données sensibles transmises et éviter les attaques par interception de données.

7. Utiliser des listes blanches de domaines autorisés

Utiliser des listes blanches de domaines autorisés pour éviter les appels HTTP vers des URLs contrôlées par l'utilisateur et éviter les attaques par SSRF.

8. Utiliser des index pour améliorer les performances

Utiliser des index sur les colonnes utilisées dans les requêtes SQL pour améliorer les performances. Cela peut être fait en ajoutant des index sur les colonnes nécessaires dans la base de données.

9. Utiliser des structures de données pour améliorer les performances

Utiliser des structures de données comme des listes ou des dictionnaires pour stocker les données et améliorer les performances. Cela peut être fait en remplaçant les boucles imbriquées par des structures de données.

10. Utiliser des mécanismes de cache pour améliorer les performances

Utiliser des mécanismes de cache pour stocker les résultats de requêtes SQL ou de calculs pour améliorer les performances. Cela peut être fait en utilisant des bibliothèques de cache comme Redis ou Memcached.

11. Ajouter des docstrings pour les fonctions

Ajouter des docstrings pour expliquer le rôle de chaque fonction, notamment `get_db()`, `get_database_session()`, `get_projects()`

12. Améliorer la lisibilité du code

Réorganiser le code pour améliorer la lisibilité et la compréhension de chaque fonction

13. Utiliser une base de données transactionnelle

Il est recommandé d'utiliser une base de données transactionnelle pour garantir la cohérence des données en cas d'erreur. Cela peut être réalisé en utilisant les transactions de SQLAlchemy.

14. Utiliser des exceptions spécifiques

Il est recommandé d'utiliser des exceptions spécifiques pour fournir des informations utiles à l'utilisateur en cas d'erreur. Cela peut être réalisé en utilisant les exceptions de SQLAlchemy.

15. Utiliser des tests unitaires

Il est recommandé d'utiliser des tests unitaires pour valider la fonctionnalité de chaque module de l'application. Cela peut être réalisé en utilisant les frameworks de test tels que Pytest ou Unittest.

16. Réduire la complexité des fonctions

Réduire la complexité des fonctions en utilisant des fonctions plus petites et plus faciles à comprendre. Cela améliorera la lisibilité et la maintenabilité du code.

17. Utiliser des noms de variables significatifs

Utiliser des noms de variables plus significatifs et plus faciles à comprendre. Cela améliorera la lisibilité et la maintenabilité du code.

18. Réduire la duplication de code

Supprimer la duplication de code en utilisant des fonctions plus petites et plus faciles à comprendre. Cela améliorera la lisibilité et la maintenabilité du code.

19. Utiliser des jointures pour réduire le nombre de requêtes

Les jointures permettent de réduire le nombre de requêtes à la base de données, ce qui peut améliorer les performances. Par exemple, au lieu de faire une requête pour récupérer tous les dépôts et une autre requête pour récupérer les informations des propriétaires, vous pouvez faire une seule requête qui joint les deux tables.

20. Utiliser un mécanisme de cache pour les données rarement modifiées

Les mécanismes de cache peuvent aider à améliorer les performances en stockant les données qui ne changent pas souvent. Par exemple, si vous avez une page qui affiche les informations d'un utilisateur, vous pouvez stocker les informations de l'utilisateur dans un cache pour éviter de faire une requête à la base de données à chaque chargement de la page.

21. Utiliser un gestionnaire de contexte pour les ressources

Les gestionnaires de contexte peuvent aider à gérer les ressources et à éviter les erreurs de mémoire. Par exemple, si vous avez un fichier ouvert, vous pouvez utiliser un gestionnaire de contexte pour fermer le fichier automatiquement lorsque vous n'en avez plus besoin.

22. Utiliser des boucles avec complexité $O(n)$ ou moins

Les boucles avec complexité $O(n)$ ou moins peuvent aider à améliorer les performances en évitant les boucles imbriquées qui peuvent être coûteuses en termes de temps de calcul. Par exemple, au lieu de faire une boucle imbriquée pour parcourir une liste de dépôts, vous pouvez utiliser une boucle avec complexité $O(n)$ pour parcourir la liste.

23. Ajouter des commentaires explicatifs

Ajouter des commentaires explicatifs pour expliquer la logique des fonctions et des variables

24. Renommer les variables et les fonctions

Renommer les variables et les fonctions pour les rendre plus explicites et faciles à comprendre

25. Ajouter des tests unitaires

Ajouter des tests unitaires pour valider le bon fonctionnement des fonctions et des variables

26. Réduire la complexité de la fonction `handleSubmit`

La fonction `handleSubmit` est trop longue et complexe. Il est recommandé de la diviser en plusieurs fonctions plus petites et plus faciles à comprendre. Cela améliorera la lisibilité et la maintenabilité du code.

27. Utiliser des noms de variables plus significatifs

Les noms de variables utilisés dans le code sont trop courts et ne sont pas très significatifs. Il est recommandé d'utiliser des noms de variables plus longs et plus explicites pour améliorer la lisibilité du code.

28. Utiliser un fichier de configuration séparé pour les options de Next.js

Il est recommandé d'utiliser un fichier de configuration séparé pour les options de Next.js. Cela améliorera la lisibilité et la maintenabilité du code.

29. Utiliser des types pour les options de Next.js

Il est recommandé d'utiliser des types pour les options de Next.js. Cela améliorera la lisibilité et la maintenabilité du code.

30. Utiliser une bibliothèque de gestion d'authentification sécurisée

Utilisez une bibliothèque de gestion d'authentification sécurisée, comme Next-Auth, pour gérer l'authentification et l'autorisation de vos utilisateurs. Cela vous permettra de configurer les jetons JWT avec une expiration et un algorithme sécurisé, ce qui réduira les risques d'attaque par injection de jetons.

31. Valider les données entrantes

Avant de les utiliser dans la requête de réinitialisation du mot de passe, validez les données entrantes pour éviter les attaques par injection de données. Cela inclut la validation des champs email, OTP et mot de passe pour s'assurer qu'ils sont corrects et ne contiennent pas de caractères malveillants.

32. Activer la configuration de sécurité pour Next.js

Activer la configuration de sécurité pour Next.js pour protéger votre application contre les attaques de brute force et valider les en-têtes HTTP de sécurité. Cela inclut la configuration de la protection contre les attaques de brute force, la validation des en-têtes HTTP de sécurité et la configuration de la sécurité pour les requêtes entrantes.

33. Optimisation de la requête SQL

Utiliser les colonnes nécessaires au lieu de `SELECT *` pour améliorer les performances de la requête SQL. Exemple : `SELECT email, otp, password FROM table WHERE email = :email`

34. Amélioration de la recherche dans une collection

Utiliser un algorithme de recherche plus efficace ou un index/dictionnaire pour améliorer les performances de la recherche dans une collection. Exemple : Utiliser un Map ou un Set pour stocker les éléments de la collection

35. Gestion des fichiers ouverts

Utiliser un gestionnaire de contexte pour les fichiers ouverts pour améliorer les performances et éviter les erreurs de mémoire. Exemple : Utiliser un try/finally pour fermer les fichiers ouverts

36. Gestion des connexions réseau

Utiliser un gestionnaire de contexte pour les connexions réseau pour améliorer les performances et éviter les erreurs de connexion. Exemple : Utiliser un try/finally pour fermer les connexions réseau

37. Utiliser des commentaires pour expliquer les algorithmes

Utilisez des commentaires pour expliquer les algorithmes complexes ou non évidents. Cela aidera les autres développeurs à comprendre le code et à le maintenir.

38. Décomposer la méthode `handleSubmit`

La méthode `handleSubmit` contient plusieurs responsabilités. Il est recommandé de la décomposer en méthodes plus petites et plus spécifiques. Par exemple, vous pouvez créer une méthode pour valider les données, une autre pour envoyer la requête API, etc.

39. Supprimer le fichier next.config.ts

Le fichier next.config.ts n'est pas utilisé dans le code. Il est recommandé de le supprimer ou de le remplacer par un fichier de configuration plus pertinent.